

CLU-ID: Contrastive learning based unsupervised intrusion detection for low-power IoT devices

Yi Chen^a, Wenxi Liu^b, Xu Yang^{c,d,*}, Wencheng Yang^d

^a Key University Laboratory of Fujian Province Digitalized Cyber Crime Supervision Prevention Control and the Collaborative Innovation Research Center of Intelligent Policing, Fujian Police College, Fuzhou, China

^b College of Computer and Data Science, Fuzhou University, Fuzhou, China

^c Fuzhou Institute of Oceanography, School of Cyber Science and Engineering, Minjiang University, Fuzhou, China

^d School of Mathematics, Physics and Computing, University of Southern Queensland, Queensland, Australia

ARTICLE INFO

Keywords:

IoT
Low-power devices
Contrastive learning
Unsupervised learning
Intrusion detection

ABSTRACT

The proliferation of Internet of Things (IoT) devices raises significant security concerns, yet existing anomaly detection methods often overlook the data scarcity of low-power devices and the computational constraints of IoT gateways. This paper addresses these challenges by first analyzing traffic samples with t-SNE and Kernel Density Estimation, revealing that samples from other devices exhibit similarity to attack samples of a target device. Based on this observation, a novel contrastive learning-based unsupervised intrusion detection method named CLU-ID is proposed, which operates in two stages: contrastive pre-training and centroid-centered training. The contrastive pre-training stage employs a contrastive loss to distinguish target device flows from others, directing the model to focus on distinctive features of the target device rather than common features shared across devices. The centroid-centered training stage models the normal distribution of each device within a single neural network, generating central vectors that represent typical flow features and enabling anomaly detection as distribution outliers. Experimental results demonstrate that CLU-ID achieves state-of-the-art performance with limited training samples, outperforming Autoencoder (AE), Deep SVDD (DSVDD), Kitsune, and six traditional methods across both shared and individual model scenarios. Furthermore, the model significantly reduces parameter volume compared to AE and DSVDD, enhancing suitability for resource-constrained gateways, while also exhibiting effective detection capability for unseen attacks.

1. Introduction

In recent years, the exponential proliferation of Internet of Things (IoT) devices across a myriad of applications has ushered in a transformative era of connectivity and convenience. However, this pervasive integration of IoT technology has also raised substantial concerns regarding the security and privacy of the data transmitted and processed by these devices. As a consequence, IoT intrusion detection has emerged as a paramount research area.

Recently, significant progress has been made in IoT intrusion detection. For instance, Saba et al. [1] propose an Intrusion Detection System (IDS) method based on a Convolutional Neural Network (CNN), which demonstrates the ability to detect potential intrusions and anomalous traffic behaviors, achieving high accuracy on the dataset. Otoum et al. [2] introduce an IDS based on Spider Monkey Optimization (SMO) and Stacked Deep Polynomial Networks (SDPNs), capable of detecting multiple types of attacks and showing improved detection performance. However, the deep networks employed by these methods

have excessive parameters, making them challenging to deploy on gateways and programmable switches [3,4].

In gateway-based IoT intrusion detection, lightweight design is critical for practical deployment [5,6]. Due to hardware cost limitations, IoT gateways typically have limited computing resources for packet forwarding. For example, the TP-Link Archer C20, a common gateway based on the Mediatek MT7620 chipset, features a MIPS architecture processor with a clock frequency of 580 MHz, along with only 8 MB of flash memory and 64 MB of RAM. These constrained resources make it difficult for most gateways to accommodate complex detection models. A feasible solution involves executing the training phase on cloud servers while maintaining reduced computational load during inference to adapt to resource-constrained gateways.

Moreover, the cost of collecting attack samples and the insufficient samples from low-power devices also limit method applicability. Firstly, network traffic features from the same attack program can vary

* Corresponding author.

E-mail addresses: chenyi@fjpsc.edu.cn (Y. Chen), wenxiliu@fzu.edu.cn (W. Liu), xu.yang@mju.edu.cn (X. Yang), wencheng.yang@unisq.edu.au (W. Yang).

<https://doi.org/10.1016/j.csi.2026.104156>

Received 28 June 2025; Received in revised form 11 February 2026; Accepted 3 March 2026

Available online 7 March 2026

0920-5489/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

across different scenarios, making it often ineffective to directly train models for a target IoT system with attack samples from public datasets. However, the cost of sampling attack traffic for each individual network scenario is significant. Therefore, an increasing number of studies have begun to employ unsupervised anomaly detection methods. Secondly, due to the low-power characteristics of IoT devices, certain devices often generate only a limited number of communication samples. For instance, in CICIOT2022 [7], the D-Link DCHS-161 Water Sensor generates only 0.6 samples per hour. This limited sample size presents challenges for unsupervised learning methods.

To address the aforementioned challenges, this work explores the potential value of distinguishing between IoT device traffic samples by observing their distribution characteristics. It is hypothesized that samples from other devices may share similarities with attack samples of a target device, due to common programming frameworks and rules. To validate this hypothesis, the t-Distributed Stochastic Neighbor Embedding (t-SNE) [8] is employed for dimensionality reduction and visualization, revealing significant overlap between other device samples and attack samples, thereby confirming the hypothesis. Additionally, using Kernel Density Estimation (KDE) [9], substantial overlap is identified in kernel distribution curves, further supporting this belief. These findings enable the leveraging of contrastive pre-training tasks, enhancing model performance by incorporating additional information from other devices.

Based on the sample distribution characteristics, this work proposes CLU-ID, an efficient contrastive learning-based unsupervised intrusion detection method for low-power IoT devices, consisting of two stages: Contrastive Pre-training and Centroid-centered Training. Firstly, a MLP-based encoder is trained to efficiently extract the discriminative features for a specific IoT device. A contrastive loss is incorporated to extract the features of the target device's normal flows by distinguishing between the normal flows and the flows from other devices, which prompts the model to focus on the distinctive features of IoT device flows rather than the common features shared across flows. Secondly, in the encoder, the distribution of the respective normal samples for every device is modeled within a single MLP network and their features are coherently fitted to this distribution to obtain a set of robust representations. Consequently, each device generates a "central vector" that represents the most typical features of its flows. Therefore, outliers of the learned distribution can be treated as abnormal flows. Since the proposed method employs a small MLP network and does not require a decoder to reconstruct the samples, it effectively reduces the computational load and makes this method more suitable for resource-constrained gateways. This unsupervised paradigm fundamentally differs from existing supervised contrastive methods. Unlike methods such as TrafCL [10] and SmartDetector [11], which are designed for Internet intrusion detection and require malicious samples during either pre-training or fine-tuning. CLU-ID performs contrastive learning solely through the distribution patterns observed across IoT device traffic flows, thereby eliminating the need for malicious training samples.

In order to evaluate the effectiveness of the proposed method, a series of comparative evaluations is conducted between CLU-ID, DSVDD, AE, Kitsune, and six traditional anomaly detection methods on three independent datasets (i.e., CICIOT2022, CICIOT2023, IoT-23). CLU-ID demonstrates the state-of-the-art performance in intrusion detection with limited training samples, achieving up to 30.17% improvement over Autoencoder (AE) and 25.58% over Deep Support Vector Data Description (DSVDD) in a shared model scenario, and up to 6.98% improvement over AE and 4.08% over DSVDD when training a model for each device. Notably, with just 200 training samples, CLU-ID achieves an area under the receiver operating characteristic curve (ROC-AUC) of 94.62%, 92.21%, and 93.53% across the three datasets, highlighting its effectiveness in limited data scenarios and mitigating performance loss associated with model sharing.

Moreover, CLU-ID significantly reduces the volume of model parameters, thereby reducing the space occupied on Flash memory. Suppose there are ten devices in an IoT environment where the TP-Link Archer C20 serves as the gateway. Based on the size of the saved model parameter files, since the AE and DSVDD methods require training a separate model for each device, these models respectively require about 700 KB and 350 KB of space, occupying 8.5% and 4.2% of the device's Flash memory. In contrast, CLU-ID trains a single shared model for all devices, which occupies about 35 KB, saving over 90% of Flash space compared to AE and DSVDD.

In summary, the contributions of this paper are as follows:

- This work explores the potential value of distinguishing between IoT device traffic samples by observing their distribution characteristics. The results from t-SNE and KDE reveal similarities between samples from different devices and those from attacks. This discovery enables contrastive pre-training to enhance model performance by leveraging the valuable insights derived from different devices.
- Unlike the commonly used method of training a separate network for each device, CLU-ID allows all devices to share a single MLP network, which significantly reduces the model's parameter count. In an IoT environment with ten devices, CLU-ID can save over 90% of the parameters compared to the AE and DSVDD methods.
- To extract features for all devices within a single network, this work proposes CLU-ID, a novel contrastive learning-based unsupervised intrusion detection method that distinguishes between normal samples from different IoT devices, enabling the acquisition of more representative features and addressing the challenge of insufficient training samples for low-power devices.
- A set of comparative evaluations is conducted between CLU-ID, DSVDD, AE, Kitsune, and six traditional anomaly detection methods on three independent datasets to validate the advantages of CLU-ID for low-power IoT devices with limited samples.
- Cross-dataset evaluation demonstrates CLU-ID's capability in detecting unseen attacks, with models trained on normal samples from one dataset successfully identifying attacks from five distinct datasets while outperforming baseline metrics.

Roadmap. The remainder of the paper is organized as follows: Section 2 outlines the motivating scenario and its challenges. Section 3 discusses the hypothesis regarding sample distribution characteristics and further details the process of hypothesis validation. Section 4 provides details on the proposed unsupervised method, which is based on contrastive learning. Section 5 presents the measurement study along with the experiment results. Section 6 reviews related prior research. Section 7 concludes the paper and discusses the limitations and potential future research.

2. Problem statement

2.1. Overall system model

The IoT system has numerous application scenarios, with smart homes being a common example. Its system model is illustrated in Fig. 1, which contains three types of entities: IoT devices, IoT gateways, and remote cloud servers. IoT devices, such as smart bulbs, automatic gates, and temperature and humidity sensors, are used to automatically sense the environment and perform operations. An IoT gateway provides a wireless access point, enabling IoT devices to connect to the internet. Remote cloud servers host IoT device management programs that assist in the operation of IoT devices. These programs collect data reported by IoT devices, issue control commands, and provide users with an interface for device management.

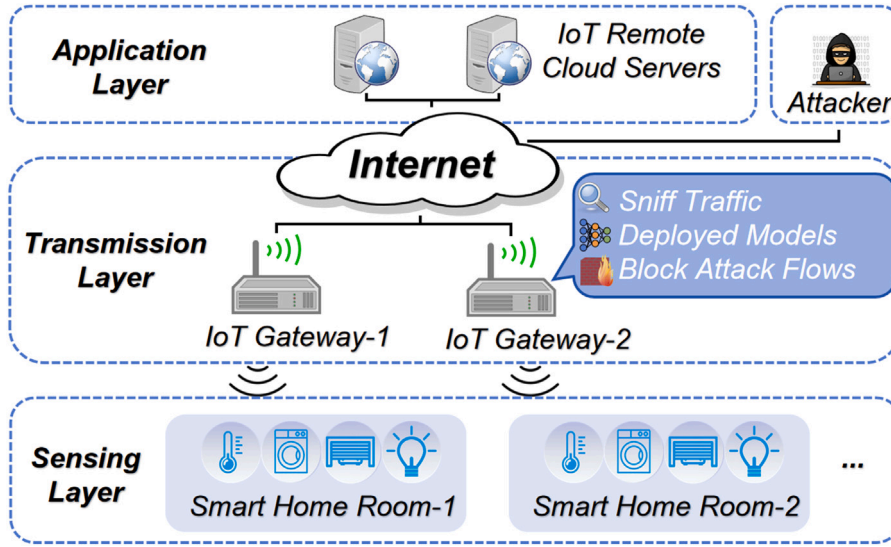


Fig. 1. System model.

2.2. Threat model

It is assumed that the IoT devices are resource-constrained and may be compromised by attackers, which could lead to engaging in malicious activities. Conversely, the IoT gateway and the remote cloud server are treated as honest entities, due to their mature security measures.

IoT Devices: IoT devices, such as smart bulbs, automatic gates, and smart washing machines, can potentially be controlled by attackers once compromised. This vulnerability may allow them to execute attacks such as injection, denial-of-service (DoS), and they may also spread malware to other devices within the network.

IoT Gateway: It is assumed that the IoT gateway is honest due to its status as a widely deployed, mature network device equipped with mature security mechanisms, making it less susceptible to compromise by malicious actors.

Remote Cloud Server: Similarly, the remote cloud server is regarded as honest because it is typically hosted in an environment with dedicated security devices and professional administrators, thus reducing the likelihood of compromise by malicious actions.

2.3. Design goals

Low hardware requirements. Given that low-cost access gateways are widely deployed in IoT environments and typically feature constrained computational and storage resources, the proposed method is designed with the training phase executed on IoT remote servers, while the inference phase maintains reduced resource consumption to ensure stable operation on existing resource-constrained gateways. Current gateways primarily include universal wireless gateways supporting TCP/IP forwarding and protocol-specific gateways for standards like Zigbee/Z-Wave. The proposed method focuses on IP-based traffic analysis, thus being most suitable for universal wireless gateways.

Suitable for low-power IoT devices. Certain low-power IoT devices typically yield only a limited number of samples during their collecting periods, making it difficult for many methods to learn the distribution characteristics of these samples, resulting in suboptimal detection effectiveness. Considering the Weakest Link Principle in network security, the proposed method is designed to provide protection for such devices to prevent them from becoming a weak point in system security.

Avoid collecting attack samples. Since the deployed IoT devices differ from those in the dataset, making direct use of attack samples from the dataset for training often ineffective. However, the cost of deploying

an attack environment and sampling in the current environment is high. Therefore, the proposed method is designed to avoid collecting attack samples through unsupervised learning techniques.

2.4. Challenges

Computational complexity. Since the model needs to run on gateways directly, the model's computational process must not be overly complex. This limitation restricts the choice of model architecture, so a classical and effective MLP is adopted to construct the model, achieving satisfactory results.

Parameter count. Given the restricted storage capacity on gateways (for instance, the TP-Link Archer C20 provides only 8 MB of FLASH space), an excessive number of parameters would hinder the model's storage on the gateway. Therefore, the proposed method trains a single shared model for all devices, which significantly reduces the model's parameter count.

Insufficient training samples. Since unsupervised learning methods only learn features from normal samples, the limited number of samples from low-power devices challenges the effectiveness of unsupervised learning methods. Therefore, a contrastive learning-based method is proposed, enabling the acquisition of more representative features and addresses this challenge.

3. Observation of samples

3.1. Theoretical analysis and hypothesis

In supervised learning methods, the comparison between different labeled samples provides important information for the model. However, in unsupervised learning, there are no comparable samples, thereby increasing the difficulty of model training. Nonetheless, in IoT scenarios, there are traffic samples from other devices in addition to those of the target device. Therefore, to alleviate the challenge of insufficient samples, the proposed method seeks to maximize the utilization of available data. Suppose there are N devices in the environment, denoted as $\{n_1, n_2, n_3, \dots, n_N\}$, and that device n_i needs to be protected by an intrusion detection method. By observing the data, a hypothesis is proposed that there may be some similarity between samples from other devices $\{n_j \mid j \neq i\}$ and the attack samples for device n_i , which allows samples from other devices to serve as a partial substitute for attack samples in comparison.

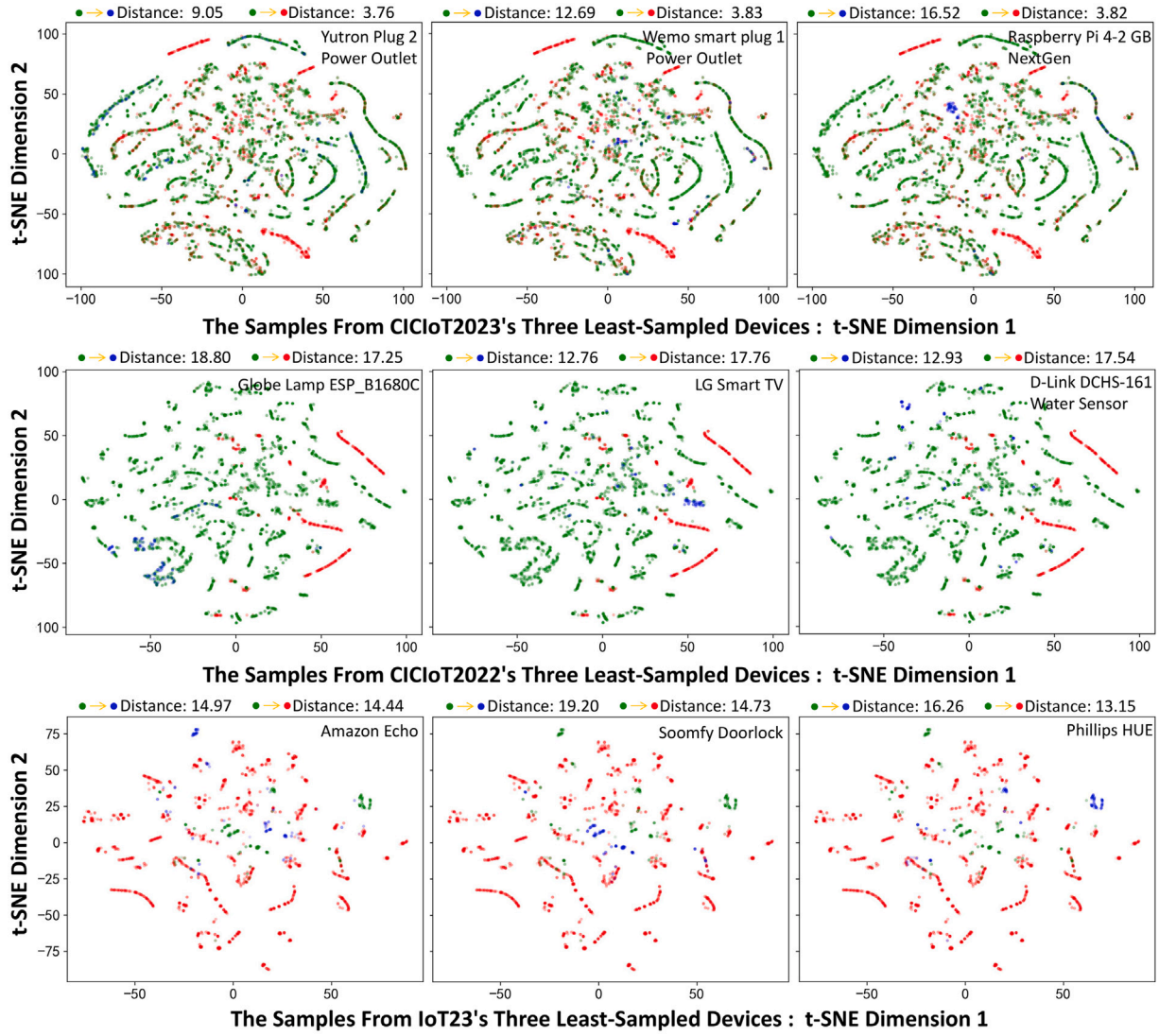


Fig. 2. T-Distributed Stochastic Neighbor Embedding (t-SNE) visualization of sample distribution from CICIoT2023, CICIoT2022, and IoT-23: normal traffic (green), attack samples (red), and a given device (blue). Nearest Neighbor Distance Statistics are introduced to quantify the similarity between two sample categories. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

This hypothesis is grounded in the following facts. Both normal and attack traffic are generated by computer programs, which typically employ similar frameworks and rules during programming. For example, many programs use the same underlying network operation libraries and default configurations, reference similar template code, and use common state machines and event-driven mechanisms. Therefore, although there are differences in the intended use between normal and attack programs, their software structures are not entirely different, and their traffic exhibits similarities. To validate this hypothesis, corresponding experiments are designed.

3.2. Hypothesis validation using t-SNE

CICIoT2023 [12] dataset is a real-time dataset and benchmark for large-scale attacks in IoT environments. The CICFlowMeter tool was employed to extract 76-dimensional sample vectors from the raw network traffic.

In the first experiment, dimensionality reduction is performed using t-SNE (from 76 to 2 dimensions) to visualize sample distributions. A stratified random sampling strategy is employed to select 200 instances per device type and attack type from the CICIoT2023 dataset. The t-SNE embedding is computed with perplexity 30, learning rate 200,

1000 iterations, random initialization, and a fixed random seed of 42. As shown in the first panel of Fig. 2, samples from the three least-sampled devices in CICIoT2023 are displayed, with device-specific samples highlighted in blue, other device samples annotated in green, and attack samples marked in red.

Nearest Neighbor Distance Statistics are introduced to quantify the similarity between two sample categories A and B : for each sample in A , the distances to its k -nearest neighbors in B are computed, and reciprocally, for each sample in B , the distances to its k -nearest neighbors in A are computed. The similarity metric is then defined as the mean of these $2 \times k$ distances. The nearest-neighbor distance uses $k = 5$ and is computed on the raw t-SNE coordinates without normalization.

The distance values labeled in Fig. 2 reveal that the distances between device samples and other device samples are frequently larger than those between other device samples and attack samples. This observation holds for 56 out of 77 tested devices (72.7%) across the CICIoT2023, CICIoT2022, and IoT-23 datasets. The pattern of smaller inter-class distances between other device samples and attack samples provides empirical support for the hypothesis.

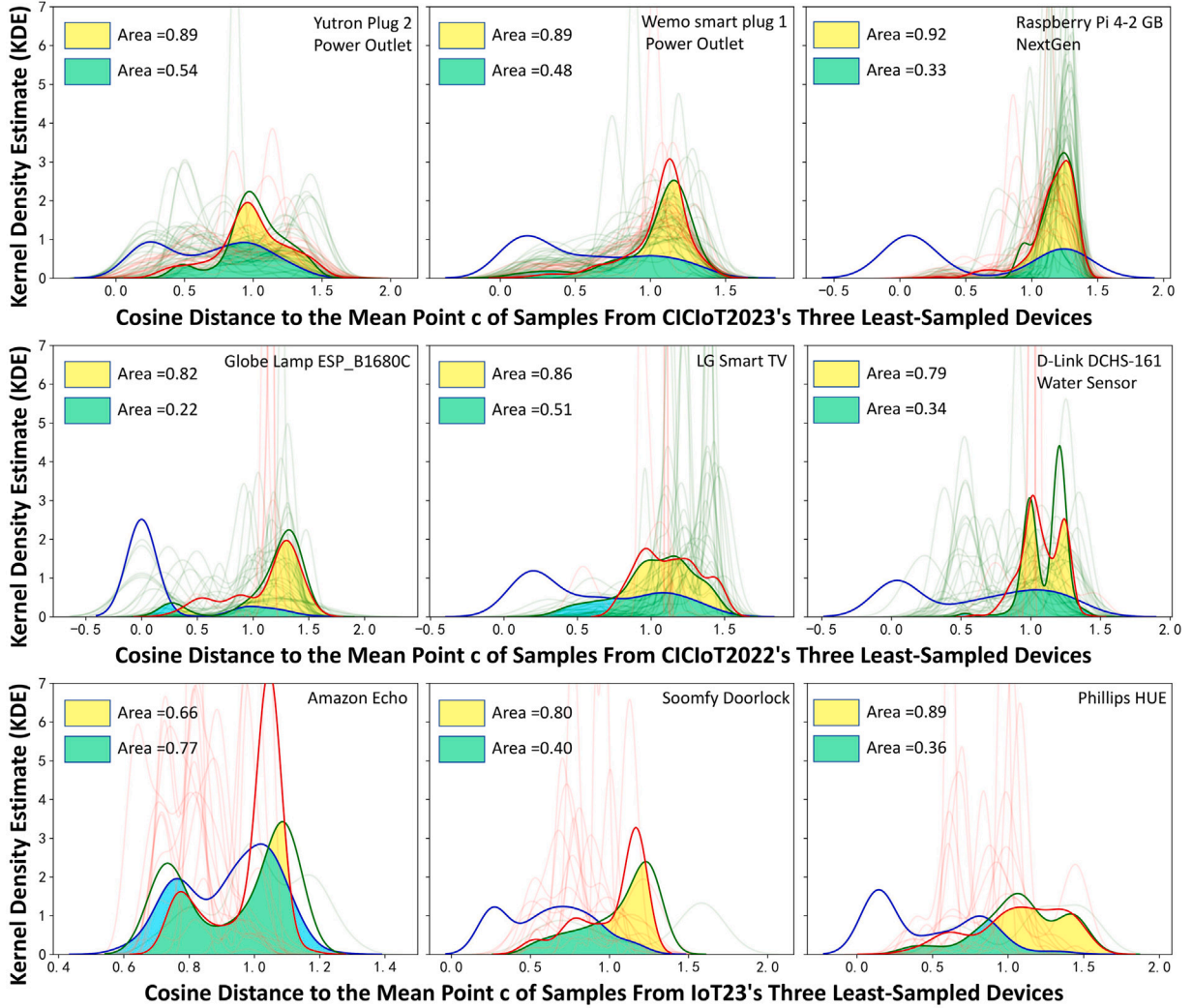


Fig. 3. Kernel Density Estimation (KDE) of cosine similarity values for multiple devices. The blue curves represent selected devices, green curves depict benign samples from other devices, and red curves indicate attack samples. Yellow-highlighted areas emphasize overlaps between benign samples from other devices and attack distributions, while cyan-highlighted areas mark overlaps between benign samples from other devices and selected device distributions. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.3. Hypothesis validation using KDE

Another method for comparing sample similarity is KDE. This method involves estimating the sample distribution through kernel smoothing, fitting a curve to approximate the distribution, and then observing the overlapping area of the fitted curves. A larger overlapping area indicates higher similarity. The vector c is obtained by averaging all samples from device n_i . Next, cosine similarity is computed between c and all remaining device samples, as well as attack samples. These cosine similarity values are then used to compute the KDE. KDE curves are generated using a Gaussian kernel with bandwidth selected by Scott's rule. The estimated densities are normalized to integrate to one, and overlap areas are computed over the intersection of the empirical supports of the two distributions. As shown in Fig. 3, samples from the three least-sampled devices in CICIOT2023, CICIOT2022, and IoT-23 are displayed. A key observation is that the significant overlap between attack distributions and benign samples from other devices (yellow areas) contrasts sharply with the reference overlaps between benign samples and selected device distributions (cyan areas).

The labeled area measurements in Fig. 3 reveal that the yellow-highlighted areas frequently exceed the cyan-highlighted reference areas. This observation holds for 76 out of 77 tested devices (98.7%)

across the CICIOT2023, CICIOT2022, and IoT-23 datasets. The prevalent pattern of larger overlapping areas between other device samples and attack samples provides quantitative support for the hypothesis.

3.4. Discussion

Experimental results indicate that, for device n_i , there may indeed be certain similarity between samples from other devices and attack samples. The key reason CLU-ID outperforms others is that its design incorporates the Contrastive Pre-training task, which leverages additional information derived from the differences between samples from various devices. The Contrastive Pre-training task in the first stage leveraged this similarity by using samples from other devices as a partial substitute for attack samples in the comparison. This approach allowed CLU-ID to utilize additional information provided by other devices in the environment, resulting in improved performance.

4. The CLU-ID methodology

Currently, IoT intrusion detection methods based on unsupervised learning typically train a separate model for each IoT device. As shown in Fig. 4(a), these methods utilize samples from each device to train a

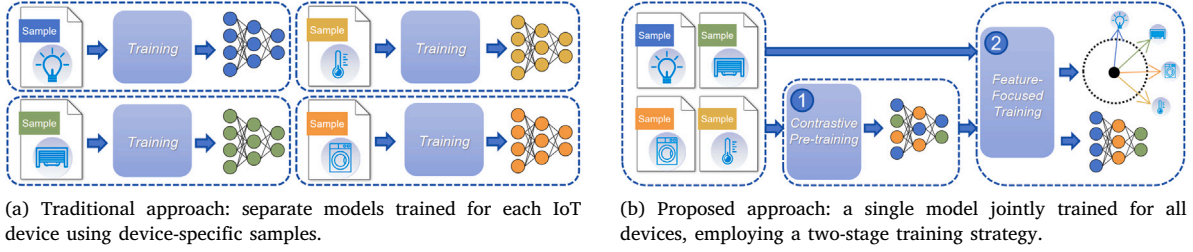


Fig. 4. Comparison of IoT intrusion detection model training strategies.

model specifically designed for that particular IoT device. During the detection phase, the gateway selects the appropriate model based on the IP packet headers of the flow to calculate the anomaly score of the samples.

However, with the increase in IoT devices, such methods consume more of the scarce disk and memory space available in IoT gateways. Therefore, CLU-ID involves jointly training a single model for all devices, as shown in Fig. 4(b). A training strategy is proposed consisting of two stages: Contrastive Pre-training and Centroid-centered Training. Firstly, in the Contrastive Pre-training stage, the contrastive loss is incorporated to extract discriminative features of normal samples from different IoT devices. Subsequently, in the Centroid-centered Training stage, the distribution of the respective normal samples is modeled and their discriminative features are fitted to that distribution, which will be applied for anomaly detection.

In summary, during the model training phase, an IDS program performs traffic sampling for all IoT devices over a period of time. Subsequently, all collected samples are preprocessed together and a shared model is trained for all devices using the processes detailed in 4.1 and 4.2. During the monitoring phase, when the program captures a traffic sample destined for device n , it performs anomaly detection using the process outlined in 4.3. If multiple anomalies are detected consecutively, it indicates that the device may be under intrusion.

4.1. Contrastive pre-training

In Contrastive Pre-training, based on the idea of contrastive learning, this method learns representations by maximizing the agreement between different data flow samples of the same IoT device through a contrastive loss function in the latent space.

The brief pre-training workflow of this method is shown in Fig. 5. Before pre-training, assuming there are N devices in the IoT environment, the training set contains data flows samples from each device. In each training epoch, the method of mini-batch gradient descent is used to train the model, where each mini-batch sample set $\{x_k\}$ consists of $2N$ data flow samples randomly selected from the training set, with each device having two samples.

In the first step, two sample vectors x_i and x_j , both derived from the same IoT device, are selected. Then, a neural network $f(\cdot)$ is employed to extract representation vectors z_i and z_j from the data flow samples, mapping these vectors to the latent space where contrastive loss is subsequently applied. This method allows various choices of the network architecture without any constraints.

In the second step, a contrastive loss function is formulated to guide a contrastive prediction task. For a target device, in the mini-batch sample set $\{x_k\}$, its two data flow samples x_i and x_j form positive sample pairs, while the other sample pairs from the other devices are regarded as background flows and used as negative examples for the target device. Given a set $\{z_k\}$ comprising a set of positive pair of representation vectors z_i and z_j , which is extracted from $\{x_k\}$ by $f(\cdot)$, the contrastive prediction task's objective is to discern z_j within $\{z_k\}_{k \neq i}$ for a given z_i . Defining the similarity function as $\text{sim}(u, v) = \cos(u, v)$,

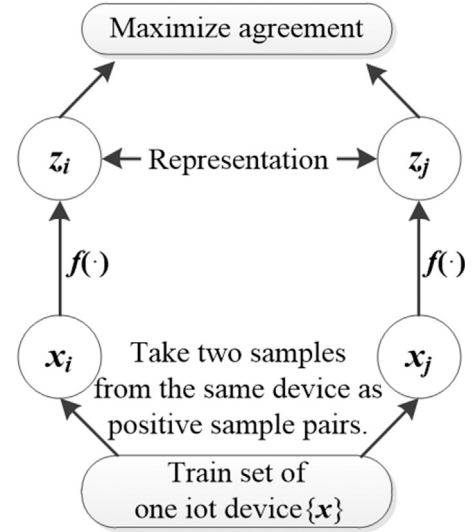


Fig. 5. Workflow of contrastive pre-training for IoT device representation learning. The process begins with sampling pairs of data flows from multiple IoT devices. For each device, two samples (x_i, x_j) are selected and encoded into representation vectors (z_i, z_j) via a neural network $f(\cdot)$. A contrastive loss function is then applied to maximize the similarity between representations from the same device while minimizing similarities across different devices. This enables the model to learn discriminative features for device identification.

the loss function for a positive pair of representation vectors (z_i, z_j) is thus formulated as

$$\ell(z_i, z_j) = -\log \frac{\exp(\cos(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{k \neq i} \exp(\cos(z_i, z_k)/\tau)}, \quad (1)$$

where $\mathbb{1}_{k \neq i} \in \{0, 1\}$ denotes an indicator function that evaluates to 1 if and only if $k \neq i$, and τ signifies a temperature parameter. The ultimate loss is then computed over all positive pairs, encompassing both (z_i, z_j) and (z_j, z_i), within a mini-batch. This loss has been used in previous work SimCLR [13].

Algorithm 1 summarizes the proposed method. The purpose of this process is to minimize the cosine distance between representation vectors of the target device, while maximizing the cosine distance between the target device's vectors and background traffic's vectors. This encourages the model to focus on learning the differential features between IoT device-oriented normal flows and background flows, rather than learning the common features that exist universally in the environment.

The computational complexity of Algorithm 1 is dominated by contrastive loss calculation. Let N denote the number of IoT devices and Z be the fixed dimensionality of latent representations. The contrastive loss computation necessitates $O(2N \times (2N - 1)) = O(4N^2 - 2N)$ operations for the $2N$ samples in each mini-batch. Although the MLP

Data: Number of devices N , Training set $\{\mathbf{x}_k\}_{k=1}^{2N}$ (2 samples per device).

Result: Trained encoder network $f(\cdot)$.

```

foreach mini-batch  $\{\mathbf{x}_k\}_{k=1}^{2N}$  do
  for  $i = 1$  to  $2N$  do
     $\mathbf{z}_i \leftarrow f(\mathbf{x}_i)$ ; // Compute latent representation  $\{\mathbf{z}\}$ 
  end
   $\mathcal{L} \leftarrow 0$ ; // Initialize total contrastive loss
  for  $k = 1$  to  $N$  do
     $i \leftarrow 2k - 1$ ; // First sample index from device  $k$ 
     $j \leftarrow 2k$ ; // Second sample index from device  $k$ 
     $\mathcal{L} \leftarrow \mathcal{L} + \ell(\mathbf{z}_i, \mathbf{z}_j) + \ell(\mathbf{z}_j, \mathbf{z}_i)$ ; // Accumulate loss
  end
   $\mathcal{L} \leftarrow \mathcal{L} / (2N)$ ; // Normalize by total samples
  Update  $f(\cdot)$  to minimize  $\mathcal{L}$ ; // Adjust model parameters
end
return encoder network  $f(\cdot)$ 

```

Algorithm 1: Contrastive Pre-training for Single Mini-batch

encoder adds $O(2N \cdot C_{\text{MLP}})$ complexity (C_{MLP} being fixed per-sample cost), these linear terms become negligible compared to the $O(N^2)$ components as N increases. Consequently, with constant Z , the overall complexity remains $O(N^2)$.

4.2. Centroid-centered training

After the Centroid-centered Training, the method aims to achieve the effect shown in Fig. 6 for the representation vectors. The method uses normal data flow samples from all devices to train the neural network $f(\cdot)$. For each IoT device n , the Centroid-centered Training goal is to minimize the sum of the cosine distances between the representation vectors and the centroid vector c_n . This allows the cosine distance between an unknown sample's representation and c_n to be used as an anomaly score in the Model Inference phase. The Centroid-centered Training process consists of two steps. Firstly, calculate the centroid vector c_n for each IoT device n . The method utilizes the $f(\cdot)$ generated in the Contrastive Pre-training stage to retrieve all training samples from the target device n and obtain a set of representation vectors $\{\mathbf{z}_n\}$ (shown in black in Fig. 6). Then, calculate the mean of all representation vectors in $\{\mathbf{z}_n\}$, and this mean vector serves as the centroid vector c_n (shown in red in Fig. 6). This process results in a set of centroid vectors $\{c\}$.

Secondly, fine-tuning the $f(\cdot)$ model. The $f(\cdot)$ is further trained to minimize the sum of the cosine distances between all representation vectors in $\{\mathbf{z}_n\}$ and the centroid vector c_n for each device n . Here, $\mathbf{x}_{n,m}$ represents the m th training sample from the target device n . Moreover, the training loss function can be defined as

$$\min_{\mathcal{W}} \frac{1}{NM} \sum_{n=1}^N \sum_{m=1}^M (1 - \cos(f(\mathbf{x}_{n,m}, \mathcal{W}), c_n)) + \frac{\lambda}{2} \sum_{\ell=1}^L \|\mathbf{w}^\ell\|_F^2. \quad (2)$$

This loss function aims to minimize the sum of the apex angles of the hypercones (shown in blue in Fig. 6) which encompass all vectors in $\{\mathbf{z}_n\}$. In Eq. (2), the first part is used to calculate the sum of cosine distances between all representation vectors and the centroid vector c_n for each device. $f(\cdot)$ represents the neural network to be trained, and $\mathcal{W}$ represents the learnable parameters of the neural network. The second part is a commonly used regularization term, aiming to add a penalty term to reduce the complexity of the model and prevent overfitting.

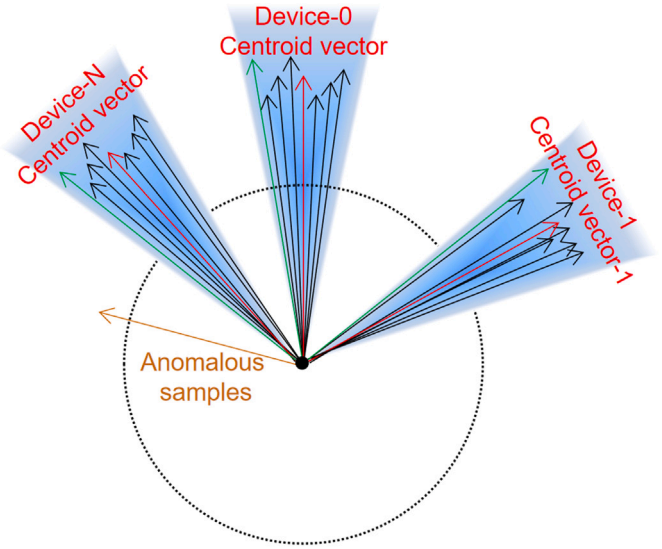


Fig. 6. Visualization after centroid-centered training: representation vectors (black) from each device are clustered around their centroid vectors (red). Blue hypercones represent the coverage regions of normal training samples. During inference, anomalous samples (yellow) appear outside these regions while normal samples (green) remain inside, enabling anomaly detection based on cosine distance to centroids. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.3. Model inference

After the model training, the model will be deployed and run on the IoT gateway. When the model receives an unknown sample $\tilde{\mathbf{x}}$ destined for device n , it generates a representation vector $\tilde{\mathbf{z}}$, and then calculates the cosine distance between $\tilde{\mathbf{z}}$ and the centroid vector c_n of device n as the anomaly score. A higher anomaly score indicates a greater difference between the features of the unknown sample and those of the target IoT device, suggesting that the sample may be a network attack data flow (shown as yellow in Fig. 6). Conversely, if the anomaly score is lower, it indicates a greater similarity between the unknown sample and a normal data flow from the target IoT device (shown as green in Fig. 6).

5. Experimental results

5.1. Dataset

IDS programs typically have the capability to capture data flows from IoT devices at the gateway. A data flow refers to a TCP or UDP session between an IoT device and a remote host. For TCP sessions, a flow begins with a SYN packet, whereas for UDP sessions, it starts with the first packet. The session concludes with a FIN, RST, or other relevant TCP packets, or based on a configured timeout period for UDP.

The method employs the CICFlowMeter tool to extract flow samples from the data flows. This tool focuses exclusively on transport layer features and excludes application layer payload content. This approach allows for the analysis of data flows with encrypted payloads or unknown application layer protocols. It extracts 76 features from the data flow samples, including duration, packet length distribution, transmission rate, flag bits, and more.

To simulate the process of capturing raw traffic flows and extracting flow samples from the gateway, three independent IoT traffic datasets are selected to provide raw PCAP capture files for experimentation, and their descriptions are as follows:

Table 1
The sample number of CICIOT2022 dataset.

Device type	Count	Device type	Count	Attack type	Count
Amazon Alexa Echo Dot 1	400 041	Gosund ESP_1ACEE1 Socket	68 796	UDPFlood	158 416
Google Nest Mini	373 321	Teckin Plug 1	65 094	HTTFFlood	141 101
Amazon Alexa Echo Spot	277 667	Amazon Plug	64 867	TCPFlood	84 955
Amazon Alexa Echo Studio	260 015	Yutron Plug 1	64 301	RTSP	28 483
BorunSichuan-AI Camera	256 730	AMCREST WiFi Camera	62 396	BruteForce	21 720
Philips Hue Bridge	235 305	SIMCAM 1S (AMPAKTec)	57 549		
Eufy HomeBase 2	219 530	HeimVision SmartLife RadioLamp	51 262		
Arlo Q Camera	183 111	Smart Board	44 213		
Home Eye Camera	181 407	Netatmo Camera	37 219		
HeimVision Smart WiFi Camera	150 830	iRobot Roomba	36 092		
Nest Indoor Camera	140 986	Luohe Cam Dog	31 024		
Ring Base Station AC_1236	135 424	Atomi Coffee Maker	30 654		
Arlo Base Station	128 829	DCS8000LHA1 D-Link Mini Camera	29 889		
Sonos One Speaker	99 646	Globe Lamp ESP_B1680C	24 376		
Gosund ESP_OC3994 Plug	92 541	LG Smart TV	16 159		
Netatmo Weather Station	78 080	D-Link DCHS-161 Water Sensor	875		

- **CICIOT2022** [7] dataset¹ is designed for feature analysis, behavior analysis, and vulnerability testing of various IoT devices across different protocols. It includes traffic samples from IoT devices in an active state over a period of 30 days, as well as traffic samples in an idle state for the same duration, totaling 1440 h, and attack traffic from multiple network attacks. Since the CICFLOW tool does not support non-IP devices like Zigbee, only devices using the TCP/IP protocol were processed. A specialized tool is developed to extract PCAP traffic sample files from the original benign traffic based on device type, using the IoT device MAC address table from the dataset. The normal traffic and various attack traffic were then processed separately using the CICFLOW tool, which enabled the generation of the sample dataset used in the experiments. The final number of samples for each device type and attack is detailed in Table 1.
- **CICIOT2023** [12] dataset² is a real-time dataset and benchmark for large-scale attacks in IoT environments. It comprises benign traffic generated by a range of IoT devices within the same internal network environment, as well as attack traffic. The sampling period for benign traffic from the devices lasted for 28 h. By utilizing the MAC address table of IoT devices provided in the dataset, traffic samples for each individual device were isolated from the benign traffic and processed with the CICFLOW tool. The processing procedure remains consistent with that of CICIOT2022. Additionally, considering that certain IoT devices, such as cameras, in the dataset may yield a significant amount of traffic samples, the sampling was limited to a maximum of 10,000 samples per device. This was followed by deduplication and labeling operations. The final count of samples for each device type and attack is detailed in Table 2.
- **IoT-23** [14] dataset³ includes multiple scenarios of IoT network traffic. Among these, three scenarios consist of normal traffic from operational IoT devices, with sampling periods of 22 h, 5 h, and 121 h respectively, while the remaining scenarios involve network traffic from infected IoT devices. Scenarios are grouped with similar types of attacks together. Subsequently, the sample dataset for the experiment was generated using a processing procedure similar to that of CICIOT2022. The final number of samples for each device type and attack is detailed in Table 3.

From the datasets mentioned above, it is observed that IoT devices with limited samples are common. For instance, in CICIOT2022, the D-Link DCHS-161 Water Sensor generates only 0.6 samples per hour; in

CICIOT2023, the Wemo smart plug 1 Power Outlet generates approximately 23 samples per hour; and in IoT-23, the Phillips HUE generates about 14 samples per hour. However, to more thoroughly evaluate the model's ability to handle scenarios with limited device samples, the number of samples accessible during training is actively controlled. This controlled sampling strategy simulates sample shortages even for IoT devices that are typically well-sampled, enabling comprehensive evaluation of the algorithm's performance across all devices.

5.2. Overview of common intrusion detection methods

To validate the advantages of CLU-ID over similar methods in intrusion detection for low-power IoT devices, a comparative evaluation was conducted with popular methods, including six traditional anomaly detection methods, AE [15], DSVDD [16], and Kitsune [17].

AE is a commonly used anomaly detection method with extensive applications in IoT intrusion detection. Investigation of related work reveals that numerous methods [15,17–22] construct detection models based on AE or its variants. Consequently, AE is included in the comparative study as a representative of this category of deep AE-based NIDS methods. For example, an MLP-based encoder is employed to compress unknown samples into low-dimensional representations, and an MLP-based decoder reconstructs these representations into approximations of original samples. The reconstruction error serves as the anomaly score. The greater the difference between unknown samples and the AE model's training samples, the more difficult the reconstruction becomes, resulting in higher anomaly scores. This category of methods trains the AE model with normal device traffic, classifying samples with high anomaly scores as attack traffic and those with lower scores as normal traffic. Moreover, Kitsune implements a detection system through KitNET, an ensemble of autoencoders with real-time feature extraction. By decomposing traffic analysis into smaller sub-models, this common baseline method achieves efficient attack detection and is included in the comparison.

DSVDD is also a common anomaly detection method that has achieved good results in image anomaly detection and can also be applied to NIDS [23,24]. Consequently, DSVDD is included in the comparative study as a representative of this category of methods. Its training process consists of two stages. Initially, an AE is used for pre-training, which enables the encoder to fit the sample features and effectively compress the training samples into low-dimensional representations. Subsequently, the encoder generates low-dimensional representations for all samples and uses the mean as the "center". The encoder is then retrained to minimize the sum of the distances between the low-dimensional representations of all training samples and this center. After generating low-dimensional representations for unknown samples through the encoder, the Euclidean distance between the representation and the center is used as the anomaly score. The

¹ Available at <https://www.unb.ca/cic/datasets/iotdataset-2022.html>.

² Available at <https://www.unb.ca/cic/datasets/iotdataset-2023.html>.

³ Available at <https://www.stratosphereips.org/datasets-iot23>.

Table 2

The sample number of CICIOT2023 dataset.

Device type	Count	Device type	Count	Attack type	Count
Arlo Q Indoor Camera Camera	9972	Yi Outdoor Camera Camera	6259	Mirai-greeth_flood	259 540
Nest Indoor Camera Camera	9915	Levoit Air Purifier Home Automation	6184	Mirai-udpplain	218 330
Amazon Echo Show Audio	9905	LG Smart TV Home Automation	6073	Mirai-greip_flood	206 471
Wyze Camera Camera	9898	Philips Hue Bridge Hub	5103	MITM-Arpspoofing	19 672
Amazon Alexa Echo Dot 1 Audio	9889	Sonos One Speaker Audio	4576	DictionaryBruteForce	9756
Netatmo Camera Camera	9849	Netatmo Weather Station Home Automation	3393	DNS_Spoofing	9753
Amazon Alexa Echo Studio Audio	9693	Teckin Plug 1 Power Outlet	3239	Recon-HostDiscovery	9752
harman kardon (Ampak Technology) Audio	9606	HeimVision SmartLife RadioLamp Lighting	2769	VulnerabilityScan	9729
SmartThings Hub Hub	9553	Amazon Plug Power Outlet	2629	SqliInjection	9715
AeoTec Smart Home Hub Hub	9540	Gosund Power strip (1) Power Outlet	2305	Recon-PortScan	9715
Home Eye Camera Camera	9413	Atomi Coffee Maker Home Automation	1564	Recon-OSScan	9636
Google Nest Mini Speaker Audio	9279	Teckin Light Strip Lighting	1489	CommandInjection	9198
TP-Link Tapo Camera Camera	9119	Govee Smart Humidifier Home Automation	1463	BrowserHijacking	9024
Yi Indoor 1 Camera Camera	9063	LampUX RGB Lighting	1284	XSS	7806
HeimVision Smart WiFi Camera Camera	8244	Lumiman bulb Lighting	1266	Backdoor_Malware	6344
Arlo Base Station Camera	8030	DCS8000LHA1 D-Link Mini Camera Camera	1229	Recon-PingSweep	4503
Rbcior Camera Camera	7845	GoSund Bulb Lighting	1195	Uploading_Attack	2977
AMCREST WiFi Camera Camera	7812	LIFX Lightbulb Lighting	1168		
Cocoon Smart HVAC Fan Home Automation	7416	Yutron Plug 2 Power Outlet	1139		
Eufy HomeBase 2 Hub	7230	Wemo smart plug 1 Power Outlet	659		
GoSund Smart plug WP2 (1) Power Outlet	6595	Raspberry Pi 4-2 GB NextGen	609		

Table 3

The sample number of IoT-23 dataset.

Device type	Count	Attack type	Count	Attack type	Count
Amazon Echo	6714	Mirai-1	12 612	Gagfyt	1639
Soomfy Doorlock	6671	Muhstik	10 601	Torii-1	1050
Phillips HUE	1781	Hide and Seek	7165	Torii-2	1032
		Trojan	3746	Linux,Mirai	847
		Mirai-2	3106	Mirai-4	801
		Mirai-3	2562	Hakai	615
		Linux.Hajime	2411	Mirai-5	472
		IRCBot	1961		

further an unknown sample is from the center, the more likely it is to be classified as an attack flow.

Six traditional anomaly detection methods are used in the experiments. K-Nearest Neighbors (KNN) [25] flags anomalies based on distances to the k nearest data points, highlighting those with distances exceeding a set threshold. Histogram-Based Outlier Score (HBOS) [26] uses feature space histograms to compute outlier scores, reflecting data point fit within the overall distribution. Local Outlier Factor (LOF) [27] identifies anomalies by comparing local densities, marking points with significantly lower densities than their neighbors. Isolation Forest (IForest) [28] creates a tree forest to isolate anomalies based on their unique attributes, leading to shorter tree paths. Local Density-Based Outlier (LODA) [29] projects data into lower dimensions and computes densities to spot outliers with relatively lower densities. Lastly, Principal Component Analysis (PCA) [30] detects anomalies by analyzing reconstruction errors in reduced-dimensional space, identifying points that deviate significantly from the projected subspace.

5.3. Configuration

Although CLU-ID allows for various choices of network architecture, to better evaluate the performance between algorithms, the approach introduced by NbaIoT is used to construct the network. In this approach, the input layer dimension of each AE equals the number of features in the dataset (i.e., 76), and the four hidden layers of the encoder are set to decreasing sizes of 75% (i.e., 57), 50% (i.e., 38), 33% (i.e., 25), and 25% (i.e., 19) of the input layer dimension. The decoder has the same dimensions as the encoder but follows an increasing order (starting from 33%) [15]. In the experiment, this approach was employed to build an AE-based detection model. To enable comparative analysis, the DSVDD source code was modified by replacing its encoder

and decoder with the same MLP network used in the AE method. Moreover, CLU-ID also utilizes the encoder network from the AE method to construct the model.

5.4. Metrics

In supervised learning-based intrusion detection, models typically output the probability that an unknown sample is an attack, so metrics such as accuracy are frequently used for evaluation. However, in unsupervised learning-based intrusion detection, the model only learns the features of normal traffic during training and does not encounter attack samples. Consequently, the model typically produces an anomaly score, where a higher score indicates a larger difference between the unknown sample and the normal samples used during training. To more directly evaluate the performance of unsupervised detection models, ROC-AUC is commonly used.

During evaluation, the test set should include both normal and attack samples. After excluding the training set samples, all remaining normal samples of each device are included in its corresponding test set. To avoid the evaluation being dominated by the large number of attack samples relative to normal traffic, a balanced test set is constructed for each device that contains all its normal samples and an equal total number of attack samples. To ensure a fair assessment across diverse attack types, the attack samples are formed by selecting the same number of instances from each attack type, so that no single attack dominates the evaluation. This design accounts for the fact that some attacks (e.g., brute-force) have abundant samples while others (e.g., Trojan) are rare but highly dangerous. This yields a binary classification task (normal versus any attack) with a 1:1 class ratio. The ROC-AUC is computed on each device-specific test set, and the final performance metric is the macro-average of the ROC-AUC values across all devices. This averaging treats every device equally, reflecting the model's overall capability in heterogeneous IoT environments.

Moreover, each model used in the experiments is trained ten times with different random seeds. For each training run, a specified number of samples is randomly selected from the normal samples to form the training set, and the test set is regenerated. The random seed is also used to initialize the neural network parameters. Ultimately, the average of the ten test results is used as the final test metric.

5.5. Comparison with state-of-the-art methods

In the first set of experiments, a comparative study was designed to evaluate the detection performance of various methods with only a

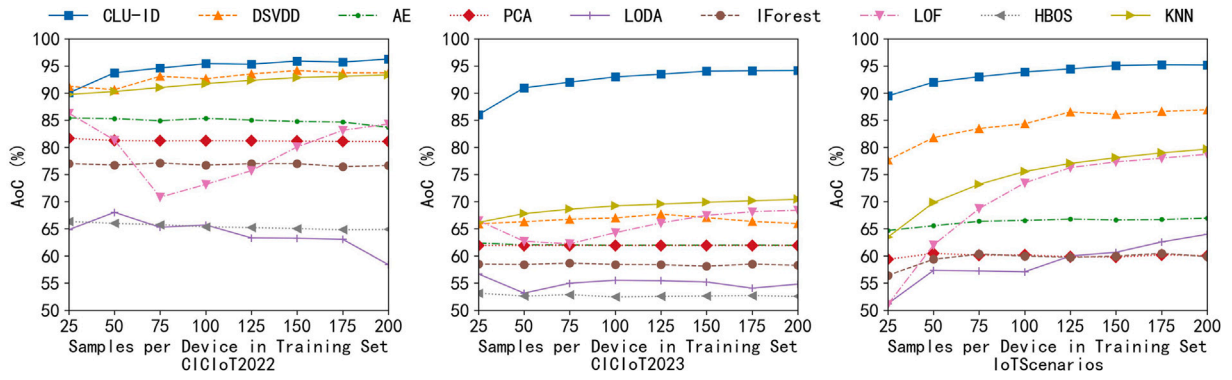


Fig. 7. Performance comparison of various methods for detection with limited training samples per device, where all devices share a single model. The ROC-AUC is used as the evaluation metric. Results are shown across three independent datasets, with the number of training samples per device (M) ranging from 25 to 200 in increments of 25.

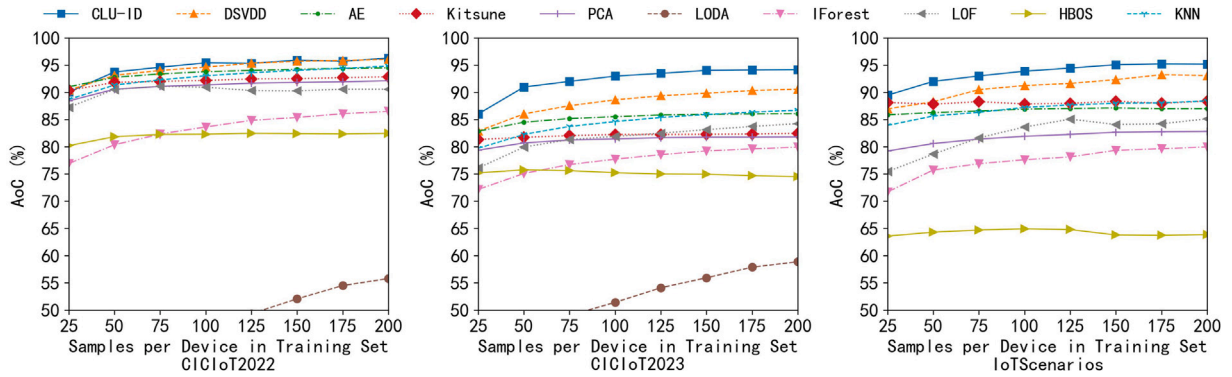


Fig. 8. Performance comparison of various methods for detection with limited training samples per device, using individual models for each device. Results are shown across three independent datasets, with the number of training samples per device (M) ranging from 25 to 200 in increments of 25.

small amount of training samples. The experiment controls the number of normal samples per device in the training set, represented by M . The values of M form an arithmetic sequence ranging from 25 to 200, with a step size of 25. Assuming there are N types of devices in the current dataset, the total number of samples in the training set is $N \times M$. Since CLU-ID involves training a shared model for all devices, other methods are also configured to train a shared model for all device samples in the first set of experiments to facilitate comparison. The Kitsune method was excluded from this experiment as it intrinsically requires training individual models for each device. The experimental results across three independent datasets are shown in Fig. 7.

In the second set of experiments, the experiments were re-conducted using the tricks from NbaIoT: for methods other than CLU-ID, each method was modified to separately train a model for each individual device. For these methods, the training set for each device's model consists of M samples, and the average of the test results across the N models is used as the final test metric. The experimental results are shown in Fig. 8.

First, from a general trend perspective, as the number of training samples increases, the model's test metric shows an upward trend. This suggests that, for the current problem, a limited number of training samples indeed restricts the model's detection performance. Once the number of training samples reaches 200, the upward trend in the metric becomes less pronounced, indicating that the model's detection capability is now primarily limiting performance improvements. Additionally, the detection performance of CLU-ID, DSVDD, and AE methods is generally superior to that of other methods, suggesting that MLP-based methods may be more appropriate for the current problem. Therefore, a more in-depth comparison of these three methods is conducted, and the results are presented in Tables 4 and 5.

As shown in Tables 4 and 5, CLU-ID achieved the highest metrics across three independent datasets. When training a shared model for all devices, CLU-ID demonstrated the maximum improvement of 25.58% over the DSVDD method, with an average improvement of 12.24%, and achieved the maximum improvement of 30.17% over the AE method, with an average improvement of 22.39%. When training a model for each device using the DSVDD and AE methods, CLU-ID achieved the maximum improvement of 4.08% over the DSVDD method, with an average improvement of 2.32%, and attained the maximum improvement of 6.98% over the AE method, with an average improvement of 4.97%. With only 200 training samples, CLU-ID achieved ROC-AUC metrics of 94.62%, 92.21%, and 93.53% on the CICIoT2022, CICIoT2023, and IoT-23 datasets, respectively. These results demonstrate that, even with the same MLP network structure, the model trained using the proposed contrastive learning-based method can more effectively address the challenge of limited training samples for low-power devices.

Comparing the two sets of experiments shows that, for both DSVDD and AE methods, training a model for each device yields significantly better results compared to training a shared model for all devices. This result further underscores the value of CLU-ID, as using either of these methods to train a shared model for all devices would lead to a loss in performance. In contrast, CLU-ID avoids such issues.

5.6. Model detection performance on unseen attacks

Given that CLU-ID is an unsupervised learning-based intrusion detection technique, it possesses the capability to detect previously unseen attack samples to some extent. To evaluate this capability of the model, a third set of experiments is designed. First, datasets are compiled using attack traffic from the CICIDS2018 and NbaIoT datasets. CICIDS2018 is a public dataset used for internet network intrusion

Table 4

Performance comparison (in percentage): CLU-ID versus DSVDD and AE methods when training a shared model for all devices. Improvements in metrics achieved by CLU-ID are shown against DSVDD and AE.

Dataset	Method	ROC-AUC (M = 25)	ROC-AUC (M = 50)	ROC-AUC (M = 75)	ROC-AUC (M = 100)	ROC-AUC (M = 125)	ROC-AUC (M = 150)	ROC-AUC (M = 175)	ROC-AUC (M = 200)	Mean	Improvement of CLU-ID
CICIoT2022	CLU-ID	90.03%	93.72%	94.61%	95.41%	95.32%	95.89%	95.70%	96.28%	94.62%	–
CICIoT2022	DSVDD	91.23%	90.62%	93.07%	92.63%	93.51%	94.15%	93.72%	93.73%	92.83%	1.79%
CICIoT2022	AE	85.40%	85.28%	84.90%	85.33%	85.01%	84.76%	84.65%	83.68%	84.88%	9.74%
CICIoT2023	CLU-ID	86.01%	90.96%	91.99%	92.97%	93.47%	94.03%	94.11%	94.16%	92.21%	–
CICIoT2023	DSVDD	65.92%	66.30%	66.76%	67.00%	67.69%	67.05%	66.36%	65.95%	66.63%	25.58%
CICIoT2023	AE	62.36%	62.05%	62.04%	61.95%	61.98%	61.98%	62.01%	61.94%	62.04%	30.17%
IoT-23	CLU-ID	89.47%	91.99%	93.00%	93.87%	94.45%	95.06%	95.22%	95.18%	93.53%	–
IoT-23	DSVDD	77.67%	81.80%	83.46%	84.33%	86.52%	86.04%	86.63%	86.90%	84.17%	9.36%
IoT-23	AE	64.69%	65.56%	66.38%	66.54%	66.80%	66.64%	66.71%	66.95%	66.28%	27.25%

Table 5

Performance comparison (in percentage): CLU-ID versus DSVDD and AE methods when training a model for each device individually. Improvements in metrics achieved by CLU-ID are shown against DSVDD and AE.

Dataset	Method	ROC-AUC (M = 25)	ROC-AUC (M = 50)	ROC-AUC (M = 75)	ROC-AUC (M = 100)	ROC-AUC (M = 125)	ROC-AUC (M = 150)	ROC-AUC (M = 175)	ROC-AUC (M = 200)	Mean	Improvement of CLU-ID
CICIoT2022	CLU-ID	90.03%	93.72%	94.61%	95.41%	95.32%	95.89%	95.70%	96.28%	94.62%	–
CICIoT2022	DSVDD	90.38%	93.11%	94.02%	94.63%	95.33%	95.69%	95.84%	96.04%	94.38%	0.24%
CICIoT2022	AE	91.00%	92.75%	93.37%	93.78%	94.02%	94.23%	94.37%	94.49%	93.50%	1.12%
CICIoT2023	CLU-ID	86.01%	90.96%	91.99%	92.97%	93.47%	94.03%	94.11%	94.16%	92.21%	–
CICIoT2023	DSVDD	82.86%	86.02%	87.53%	88.61%	89.36%	89.83%	90.32%	90.57%	88.14%	4.08%
CICIoT2023	AE	82.84%	84.50%	85.17%	85.51%	85.83%	85.94%	86.03%	86.07%	85.24%	6.98%
IoT-23	CLU-ID	89.47%	91.99%	93.00%	93.87%	94.45%	95.06%	95.22%	95.18%	93.53%	–
IoT-23	DSVDD	86.94%	88.26%	90.49%	91.23%	91.62%	92.31%	93.23%	93.07%	90.89%	2.64%
IoT-23	AE	85.86%	86.26%	86.54%	86.89%	87.02%	87.12%	86.98%	86.98%	86.71%	6.82%

Table 6

Detection performance on unseen attack samples: Comparison with Baseline Metrics. The table presents the performance of models trained on normal samples from one dataset and tested on attack samples from CICIDS2018, NBaIoT, CICIoT2022, CICIoT2023, and IoT-23 datasets.

Normal samples	CICIDS2018	NBaIoT	CICIoT2022	CICIoT2023	IoT-23	Mean	BaseLine
CICIoT2022	100.00%	100.00%	–	96.18%	95.87%	98.01%	96.28%
CICIoT2023	100.00%	100.00%	94.73%	–	94.53%	97.32%	94.16%
IoT-23	100.00%	100.00%	95.42%	95.92%	–	97.84%	95.18%

detection, from which 14 types of attack samples are extracted. NBaIoT is a public dataset for network intrusion detection in IoT environments. Considering that NBaIoT provides attack traffic generated separately for each device, 80 types of attack samples were processed from this dataset. Additionally, attack samples are separately extracted from the CICIoT2022, CICIoT2023, and IoT-23 datasets. Based on the first set of experiments, models trained with 200 samples are selected and substitute the attack samples in the test set with those sourced from these five datasets for reevaluation. The experiments are conducted ten times with varying random seeds, and the mean of the metrics is used as the final result.

As shown in Table 6, compared to the baseline metrics from the first set of experiments, the models perform better in detecting attack samples from the CICIDS2018 and NBaIoT datasets. This may be because the attack samples in these datasets not only target different objectives compared to the normal traffic in the training set but also originate from slightly different network environments, which makes it easier for the models to detect such attack samples. Furthermore, the models also demonstrate a certain level of detection capability for attack samples from the other two datasets. Nevertheless, since the models have never encountered sample features from any of the other four datasets before, achieving metrics close to the baseline suggests that the models do possess some ability to detect previously unseen attack samples.

5.7. Hyperparameter tuning

A systematic hyperparameter tuning study is conducted to evaluate the impact of key hyperparameters on detection performance. For each configuration, the model is trained following the experimental setup

in Section 5.5 with 200 samples per device, and evaluated on the CICIoT2022, CICIoT2023, and IoT-23 datasets. The reported ROC-AUC metric represents the average of ten independent runs with different random seeds.

Encoder depth. The depth of encoder layers directly influences the model's performance. To evaluate discriminative feature extraction, the encoder depth varies from 3 to 6 layers with fixed input (76) and output (19) dimensions. Hidden layer dimensions adopt NBaIoT's proportional scaling strategy. Results in Table 7 indicate that increasing encoder depth from 3 to 5 layers enhances model performance, whereas depth 6 yields no further improvement. These findings demonstrate that 5 encoder layers achieve optimal performance.

The temperature parameter. The temperature parameter τ plays a critical role in balancing the trade-off between uniformity and tolerance in contrastive learning. Larger τ values lead to softer similarity distributions, encouraging broader exploration of the embedding space, while smaller τ values sharpen the distribution, promoting stronger discrimination between positive and negative pairs. As shown in Table 7, the temperature parameter τ in the contrastive loss function (Eq. (1)) is examined across values ranging from 0.1 to 0.005. Optimal performance metrics are attained when τ is set to 0.05.

Learning Rate Optimization. The learning rate governs parameter update magnitudes during model optimization. Excessive values induce training instability, whereas insufficient values impede convergence. Experiments in contrastive pre-training (1000 epochs) demonstrate significant performance dependence on this parameter (Table 7). Quantitative analysis identifies 0.0005 as the optimal value, with measurable degradation observed at both higher (0.005) and lower (0.0001) settings. Subsequent centroid-centered training evaluations (2000 epochs)

Table 7

Hyperparameter tuning study of encoder depth, temperature parameter (τ), and Learning Rates (lr1: Contrastive Pre-training, lr2: Centroid-centered Training).

Encoder depth			Temperature (τ)		Learning rate (lr1)		Learning rate (lr2)	
#	Encoder architecture	ROC-AUC	τ	ROC-AUC	lr1	ROC-AUC	lr2	ROC-AUC
3	76,38,19	94.21%	0.1	95.14%	0.005	92.01%	0.001	94.98%
4	76,50,32,19	95.19%	0.05	95.23%	0.001	94.81%	0.0005	95.17%
5	76,57,38,25,19	95.23%	0.01	94.97%	0.0005	95.23%	0.0001	95.23%
6	76,64,48,32,25,19	95.12%	0.005	95.05%	0.0001	94.31%	0.00005	95.19%

reveal 0.0001 as the preferred learning rate, achieving peak performance metrics.

5.8. Computational resource efficiency validation

To validate the applicability of the method to resource-constrained IoT gateways, this study designs experiments through analysis of the method workflow.

For the training phase, since most gateway hardware lacks sufficient resources for model training, CLU-ID is trained on IoT remote cloud servers before deployment to the gateways. Although the training complexity exceeds traditional methods like PCA, it remains acceptable for servers, while gateways are excluded from training.

For the inference phase, experimental verification is performed on physical devices to assess real-world inference capability. The hardware platform utilizes the MT7620 MIPS-based chipset (64 MB RAM, 128 MB flash) running embedded Linux, which represents typical gateway specifications. The core inference logic is implemented in C and deployed as cross-compiled ELF executables. Computational efficiency tests demonstrate an average latency of 40.68 ms per inference over 1000 samples. Memory consumption remains stable at 736 KB Virtual Memory Size (VSZ), comprising Resident Set Size (RSS), shared libraries, and unused reserved memory, as monitored by the Linux 'ps' command. Flash storage requirements consist of a 20.6 KB ELF executable file and a 32 KB model parameters file. This resource profile maintains feasibility for standard gateways such as the TP-Link Archer C20 (64 MB RAM, 8 MB flash).

To verify the practicality of CLU-ID in resource-limited environments, hardware experiments are conducted on physical IoT gateway devices. The experimental results demonstrate that CLU-ID achieves real-time inference capability with average latency below 50 ms while maintaining memory consumption under 736 KB on MT7620-based hardware platforms, confirming its operational feasibility for typical IoT gateways. Furthermore, CLU-ID exhibits superior flash memory efficiency compared to AE and SVDD. Considering an IoT deployment with ten devices, CLU-ID requires only 32 KB flash storage for model parameters, representing over 90% reduction compared to AE (640 KB) and SVDD (320 KB) methods implementing device-specific models. This significant storage optimization stems from CLU-ID's shared model architecture that eliminates redundant parameter storage across devices while maintaining detection accuracy.

6. Related works

Network intrusion detection focuses on analyzing network traffic to identify malicious activities. Although IoT devices face unique challenges such as resource constraints, many internet NIDS techniques offer valuable insights for IP-based IoT scenarios. This section categorizes related research into the following three main paradigms.

6.1. Supervised learning-based methods

Supervised methods require large, labeled datasets for training. Deep learning methods have become mainstream in this category due to their high performance.

Deep Learning Methods. For instance, DNN-based methods are employed. Soe et al. [31] propose a sequential detection scheme for specific IoT cyber attacks, optimizing accuracy by selecting individual classifiers and activation functions for each attack type. CNN-based methods include Kan et al.'s Adaptive Particle Swarm Optimization CNN (APSO-CNN) [32], which adaptively optimizes the structural parameters of CNNs for effective intrusion detection. LSTM-based methods are also prominent. Djaidja et al. [33] develop an early NIDS using RNNs (LSTM/GRU) enhanced with attention mechanisms. This enables real-time attack classification without waiting for flow completion, thereby reducing detection delay and limiting malicious packet infiltration. Transformer-based architectures are increasingly adopted. Han et al. [34] introduce GTID, a hierarchical intrusion detection model that utilizes n-gram frequency for packet-level feature extraction and a time-aware transformer for capturing session-level temporal dependencies, effectively minimizing information loss from raw network traffic.

Generative Methods for Imbalance Mitigation. Significant differences across network environments frequently result in class imbalance issues. To address this, Ding et al. [35] present TMG-GAN, a multi-generator GAN-based framework. Their method incorporates a classifier and a cosine similarity loss to enhance the quality of generated samples and reduce inter-class overlap, reportedly improving performance on benchmark datasets.

6.2. Semi-supervised and self-supervised methods

These methods reduce the dependency on malicious labels by leveraging unlabeled data to improve generalization, thereby addressing the practical issue of annotation scarcity. Contrastive learning represents a current research focus.

Contrastive Learning Methods. These methods adopt a "pre-training and fine-tuning" paradigm: models are first pre-trained on unlabeled data via a contrastive objective to learn robust, generalized representations, which are then adapted to the NIDS task using a small set of labeled data. Shen et al. [11] design SmartDetector to learn robust traffic representations via semantic attribute matrices (SAMs) and data augmentation, reportedly achieving F1-scores and AUC above 93% against evasion attacks. Yang et al. [10] propose TrafCL for detecting encrypted malware traffic by constructing positive pairs through session augmentation, contrasting complete command-and-control sessions with their intentionally incomplete variants, and show performance improvements over baselines. Li et al. [36] propose PASS, a Pre-training Semi-Supervised framework for encrypted traffic classification. It employs contrastive pre-training on resampled data to mitigate label bias caused by class imbalance and traffic homogeneity, combined with a pseudo-label iteration strategy for utilizing unlabeled data effectively. Lin et al. [37] develop CETP, a semi-supervised contrastive pre-training framework that addresses imbalanced encrypted traffic classification. The framework mitigates label bias through pseudo-label iterations and dynamic loss-weighting, achieving notable F1-score improvements. Earlier methods also explore this direction, such as CIDS [38] and FeCo [5].

6.3. Unsupervised learning-based methods

These methods identify anomalies by modeling the behavioral patterns of normal traffic, operating entirely without malicious samples.

Traditional Anomaly Detection Methods. Featuring a small number of parameters, traditional anomaly detection methods frequently serve as baselines in NIDS. For instance, Nguyen et al. [39] propose Fed-PCA, a federated PCA framework that learns global anomaly detection models from distributed IoT device data while preserving privacy, eliminating the need for centralized raw data collection.

Deep AE-based Methods. These methods have become widely used in NIDS with the rise of deep learning. Meidan et al. [15] present N-BaIoT, a deep AE-based network anomaly detection method for IoT botnet attacks, reporting accurate detection on devices infected by Mirai and BASHLITE. Mirsky et al. [17] introduce Kitsune, an unsupervised online IDS using an ensemble of AEs (KitNET), demonstrating effectiveness even on resource-constrained devices. Telikani et al. [18] develop a cost-sensitive stacked AE (CSSAE) to handle class imbalance in IoT IDS, aiming to enhance feature learning for better detection of low-frequency attacks. Basati et al. [19] propose a lightweight Parallel Deep AE (PDAE) for IoT network intrusion detection, seeking a balance between detection accuracy and computational demands.

DSVDD-based Methods. Building upon the success of DSVDD in image anomaly detection, these methods have been adapted for NIDS. Aktar et al. [23] introduce DSVDD-CAE, combining Deep SVDD with a Contractive AE for robust anomaly detection. Wang et al. [24] present a hybrid intrusion detection method for industrial control systems, integrating an improved AE with Deep SVDD loss, and report superior performance on multiple datasets.

Deployment-Oriented Methods. These methods build upon unsupervised learning to provide continuous adaptation to changing network environments, as well as high-throughput processing. Alotaibi et al. [20] present Mateen, an adaptive DAE ensemble method that dynamically adjusts model parameters to cope with evolving benign traffic patterns. Zhao et al. [21] introduce Trident, a universal framework that reformulates fine-grained, class-incremental encrypted traffic detection as multiple one-class learning tasks, demonstrating superior performance over existing methods. Some works also explore hybrid supervised-unsupervised paradigms for efficiency. Zhang et al. [22] introduce AOC-IDS, an autonomous online intrusion detection framework. It integrates an autoencoder with a novel Cluster Repelling Contrastive (CRC) loss for learning distinctive features from limited or incremental data, employing a labor-free online framework with pseudolabeling for continual adaptation in dynamic IoT environments. Additionally, Dong et al. [3] introduce HorusEye, a high-throughput two-stage anomaly detection framework leveraging programmable switches. It combines an unsupervised data-plane stage, using an Isolation Forest (iForest)-based model for burst-level screening, with a control-plane stage employing deep flow-level analysis, aiming for scalable high-throughput and high detection accuracy.

6.4. Position of CLU-ID

In summary, real-world network environments are highly diverse, and no single method demonstrates optimal performance across all scenarios. CLU-ID targets the deployment of NIDS on resource-constrained IoT gateways, aiming to protect low-traffic IoT devices in the absence of attack samples. Therefore, generalization capability under limited samples and model size are primary considerations.

Based on the evaluation of detection performance in Section 5.5 and the evaluation of model size in Section 5.8, CLU-ID is compared with several unsupervised learning-based methods. As shown in Table 8, the advantage of CLU-ID lies in its use of contrastive learning to enhance generalization capability for low-traffic IoT devices, while resource consumption is significantly reduced by using a shared model.

Table 8

Comparison between CLU-ID and unsupervised learning-based methods.

Method	Traditional	DAE-based	DSVDD-based	CLU-ID
Small-sample generalization	Limited	Moderate	Moderate	Strong
Parameter size	Minimal (Non-learnable)	Large	Medium	Compact

Noteworthy, CLU-ID differs fundamentally from existing contrastive methods that typically require malicious samples during training. Instead, it learns representations by contrasting the behavioral patterns inherent in normal IoT device traffic flows, thereby eliminating the dependency on malicious training data.

7. Conclusion

This paper presents CLU-ID, a novel contrastive learning-based unsupervised intrusion detection method specifically designed for resource-constrained IoT gateways. The proposed method combines contrastive pre-training that focuses on distinctive features with centroid-centered training using shared MLP networks, demonstrating three key advantages. First, it achieves significant improvements in detection accuracy over baseline methods while requiring minimal training samples. Second, the shared model architecture substantially reduces flash memory requirements compared to conventional methods. Third, extensive cross-dataset validation confirms CLU-ID's effectiveness in detecting previously unseen attack patterns. These characteristics make CLU-ID particularly suitable for low-power IoT deployments where both labeled attack samples and hardware resources are severely constrained. Future research directions include extending protocol compatibility to non-IP IoT standards like Zigbee and Z-Wave, as well as developing hybrid detection frameworks that incorporate limited attack samples.

CRedit authorship contribution statement

Yi Chen: Writing – original draft. **Wenxi Liu:** Writing – review & editing, Supervision. **Xu Yang:** Writing – review & editing. **Wencheng Yang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant 62302203, the Fund for the Innovation of Policing Science and Technology of Fujian Province under Grants 2024YZ040001 and 2024Y0061, and the Science and Technology Project of Fuzhou Institute of Oceanography under Grant 2025F04.

Data availability

Data will be made available on request.

References

- [1] Tanzila Saba, Amjad Rehman, Tariq Sadat, Hoshang Kolivand, Saeed Ali Bahaj, Anomaly-based intrusion detection system for IoT networks through deep learning model, *Comput. Electr. Eng.* 99 (2022) 107810.
- [2] Yazan Otoum, Dandan Liu, Amiya Nayak, DL-IDS: a deep learning-based intrusion detection framework for securing IoT, *Trans. Emerg. Telecommun. Technol.* 33 (3) (2022) e3803, e3803 ett.3803.
- [3] Yutao Dong, Qing Li, Kaidong Wu, Ruoyu Li, Dan Zhao, Gareth Tyson, Junkun Peng, Yong Jiang, Shutao Xia, Mingwei Xu, HorusEye: A realtime IoT malicious traffic detection framework using programmable switches, in: 32nd USENIX Security Symposium, USENIX Security 23, USENIX Association, Anaheim, CA, 2023, pp. 571–588.
- [4] Ruoyu Li, Qing Li, Yu Zhang, Dan Zhao, Xi Xiao, Yong Jiang, Genos: General in-network unsupervised intrusion detection by rule extraction, in: IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, 2024, pp. 561–570.
- [5] Ning Wang, Yimin Chen, Yang Hu, Wenjing Lou, Y. Thomas Hou, Feco: Boosting intrusion detection capability in IoT networks via contrastive learning, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022, pp. 1409–1418.
- [6] Xuan-Ha Nguyen, Xuan-Duong Nguyen, Hoang-Hai Huynh, Kim-Hung Le, Re-alsguard: A lightweight network intrusion detection system for IoT gateways, *Sensors* 22 (2) (2022) 432.
- [7] Sajjad Dadkhah, Hassan Mahdikhani, Priscilla Kyei Danso, Alireza Zohourian, Kevin Anh Truong, Ali A. Ghorbani, Towards the development of a realistic multidimensional IoT profiling dataset, in: 2022 19th Annual International Conference on Privacy, Security & Trust, PST, 2022, pp. 1–11.
- [8] Laurens Van der Maaten, Geoffrey Hinton, Visualizing data using t-SNE, *J. Mach. Learn. Res.* 9 (11) (2008) 2579–2605.
- [9] Murray Rosenblatt, Remarks on some nonparametric estimates of a density function, *Ann. Math. Stat.* 27 (3) (1956) 832–837.
- [10] Xiaodu Yang, Sijie Ruan, Jinyu Li, Yinliang Yue, Bo Sun, TrafCL: Robust encrypted malicious traffic detection via contrastive learning, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 2910–2919.
- [11] Meng Shen, Jinhe Wu, Ke Ye, Ke Xu, Gang Xiong, Liehuang Zhu, Robust detection of malicious encrypted traffic via contrastive learning, *IEEE Trans. Inf. Forensics Secur.* 20 (2025) 4228–4242.
- [12] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, Ali A. Ghorbani, CICIOT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941.
- [13] Ting Chen, Simon Kornblith, Mohammad Norouzi, Geoffrey Hinton, A simple framework for contrastive learning of visual representations, in: Hal Daumé III, Aarti Singh (Eds.), Proceedings of the 37th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, Vol. 119, PMLR, 2020, pp. 1597–1607.
- [14] Sebastian Garcia, Agustin Parmisano, Maria Jose Erquiaga, IoT-23: A Labeled Dataset with Malicious and Benign IoT Network Traffic, Stratosphere Laboratory, Praha, Czech Republic, 2020.
- [15] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, Yuval Elovici, N-Balot: Network-based detection of IoT botnet attacks using deep autoencoders, *IEEE Pervasive Comput.* 17 (3) (2018) 12–22.
- [16] Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, Marius Kloft, Deep one-class classification, in: Jennifer Dy, Andreas Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 4393–4402.
- [17] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, Asaf Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, 2018, CoRR, abs/1802.09089.
- [18] Akbar Telikani, Amir H. Gandomi, Cost-sensitive stacked auto-encoders for intrusion detection in the internet of things, *Internet Things* 14 (2021) 100122.
- [19] Amir Basati, Mohammad Mehdi Faghhi, PDAE: Efficient network intrusion detection in IoT using parallel deep auto-encoders, *Inform. Sci.* 598 (2022) 57–74.
- [20] Fahad Alotaibi, Sergio Maffei, Mateen: Adaptive ensemble learning for network anomaly detection, in: Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 215–234.
- [21] Ziming Zhao, Zhaoxuan Li, Zhuoxue Song, Wenhao Li, Fan Zhang, Trident: A universal framework for fine-grained and class-incremental unknown traffic detection, in: Proceedings of the ACM Web Conference 2024, WWW '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1608–1619.
- [22] Xinchun Zhang, Running Zhao, Zhihan Jiang, Zhicong Sun, Yulong Ding, Edith C.H. Ngai, Shuang-Hua Yang, AOC-IDS: Autonomous online framework with contrastive learning for intrusion detection, in: IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, 2024, pp. 581–590.
- [23] Sharmin Aktar, Abdullah Yasin Nur, Robust anomaly detection in IoT networks using deep SVDD and contractive autoencoder, in: 2024 IEEE International Systems Conference, SysCon, 2024, pp. 1–8.
- [24] Chao Wang, Hongri Liu, Chao Li, Yunxiao Sun, Wenting Wang, Bailing Wang, Robust intrusion detection for industrial control systems using improved autoencoder and Bayesian Gaussian mixture model, *Mathematics* 11 (9) (2023) 2048.
- [25] Sridhar Ramaswamy, Rajeev Rastogi, Kyuseok Shim, Efficient algorithms for mining outliers from large data sets, *SIGMOD Rec.* 29 (2) (2000) 427–438.
- [26] Markus Goldstein, Andreas Dengel, Histogram-based outlier score (hbos): A fast unsupervised anomaly detection algorithm, 2012, pp. 59–63, KI-2012: Poster Demo Track, 1.
- [27] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, Jörg Sander, LOF: identifying density-based local outliers, *SIGMOD Rec.* 29 (2) (2000) 93–104.
- [28] Fei Tony Liu, Kai Ming Ting, Zhi-Hua Zhou, Isolation forest, in: 2008 Eighth IEEE International Conference on Data Mining, 2008, pp. 413–422.
- [29] Tomáš Pevný, Loda: Lightweight on-line detector of anomalies, *Mach. Learn.* 102 (2016) 275–304.
- [30] Mei-Ling Shyu, Shu-Ching Chen, Kanoksri Sarinnapakorn, Liwu Chang, A novel anomaly detection scheme based on principal component classifier, in: Proceedings of the Third IEEE International Conference on Data Mining, ICDM, IEEE, 2003, pp. 353–365.
- [31] Yan Naung Soe, Yaokai Feng, Paulus Insap Santosa, Rudy Hartanto, Kouichi Sakurai, A sequential scheme for detecting cyber attacks in IoT environment, in: 2019 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress, DASC/PiCom/CBDCom/CyberSciTech, 2019, pp. 238–244.
- [32] Xiu Nan, Yixuan Fan, Zhijun Fang, Le Cao, Neal N. Xiong, Dan Yang, Xuan Li, A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network, *Inform. Sci.* 568 (2021) 147–162.
- [33] Taki Eddine Toufik Djaidja, Bouziane Brik, Sidi Mohammed Senouci, Abdelwahab Boualouache, Yacine Ghamri-Doudane, Early network intrusion detection enabled by attention mechanisms and RNNs, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 7783–7793.
- [34] Xueying Han, Susu Cui, Song Liu, Chen Zhang, Bo Jiang, Zhigang Lu, Network intrusion detection based on n-gran frequency and time-aware transformer, *Comput. Secur.* 128 (2023) 103171.
- [35] Hongwei Ding, Yu Sun, Nana Huang, Zhidong Shen, Xiaohui Cui, TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection, *Trans. Info. For. Sec.* 19 (2024) 1156–1167.
- [36] Xiang Li, Juncheng Guo, Qige Song, Jiang Xie, Yafei Sang, Shuyuan Zhao, Yongzheng Zhang, Listen to minority: Encrypted traffic classification for class imbalance with contrastive pre-training, in: 2023 20th Annual IEEE International Conference on Sensing, Communication, and Networking, SECON, 2023, pp. 447–455.
- [37] Xinjie Lin, Longtao He, Gaopeng Gou, Jing Yu, Zhong Guan, Xiang Li, Juncheng Guo, Gang Xiong, CETP: A novel semi-supervised framework based on contrastive pre-training for imbalanced encrypted traffic classification, *Comput. Secur.* 143 (2024) 103892.
- [38] Yawei Yue, Xingshu Chen, Zhenhui Han, Xuemei Zeng, Yi Zhu, Contrastive learning enhanced intrusion detection, *IEEE Trans. Netw. Serv. Manag.* 19 (4) (2022) 4232–4247.
- [39] Tung-Anh Nguyen, Long Tan Le, Tuan Dung Nguyen, Wei Bao, Suranga Seneviratne, Choong Seon Hong, Nguyen H. Tran, Federated PCA on grassmann manifold for IoT anomaly detection, *IEEE/ACM Trans. Netw.* 32 (5) (2024) 4456–4471.